Invoice and receipt intake bot

An **Invoice and Receipt Intake Bot** is a practical corporate automation project: employees upload a receipt, invoice, or expense document; ImageJev/imajev classifies it, checks readability, and routes it to the correct finance workflow. The model supports one or two images, a business-record `state`, and up to eight typed questions per request, returning probabilities plus an explicit `unknown`/abstention signal. [1] [2] [3]

What the bot does

For each uploaded document, the bot answers a small set of fixed questions:

- **Document type:** invoice, receipt, bank statement, delivery note, or other.
- **Readability:** readable, partially readable, or unreadable.
- **Business fit:** does it look relevant to the submitted expense or vendor record?
- **Routing:** expenses, accounts payable, procurement, or exception queue.
- **Risk flag:** duplicate-looking document, missing visible total, or unclear vendor.

It should **not** try to fully extract amounts, tax IDs, or line items at first. Use ImageJev for triage and quality control; use OCR or a finance-system integration later for exact field extraction.

Workflow

1. Employee uploads a photo, scan, or screenshot through Streamlit.
2. The app sends the image plus optional context—employee, vendor, amount, expense category—to `/v1/systemone`.
3. ImageJev returns typed answers and probabilities.
4. Your Python logic applies business rules.
5. The app saves the result to SQLite/CSV and routes the case.

ImageJev accepts images as base64 data URLs or as a multipart upload, with a `state` payload containing the business context and a `questions` object defining the decision options. [3] [4] [1]

Question schema

```
questions = {
    "document_type": {
        "type": "choice",
        "instructions": "What type of document is shown?",
        "criteria": {
            "invoice": "A supplier invoice requesting payment",
```

```
            "receipt": "A customer or expense receipt",
            "bank_statement": "A bank account statement",
            "delivery_note": "A shipping or delivery document",
            "other": "Another document type"
        }
    },
    "readable": {
        "type": "choice",
        "instructions": "Is the document readable enough for finance processing?",
        "criteria": {
            "yes": "Key details are legible",
            "partially": "Some details are unclear",
            "no": "The document is too blurry, cropped, or dark"
        }
    },
    "has_total": {
        "type": "noul",
        "instructions": "The document visibly contains a total amount."
    },
    "matches_context": {
        "type": "noul",
        "instructions": "The document is consistent with the submitted vendor and exp
    },
    "duplicate_risk": {
        "type": "choice",
        "instructions": "Does this document appear likely to be a duplicate submissio
        "criteria": {
            "low": "No obvious duplicate indicators",
            "medium": "Some indicators suggest a possible duplicate",
            "high": "Strong indicators suggest a duplicate",
            "unknown": "Cannot determine from the available evidence"
        }
    }
}
```

Use `choice` when you need a category, `noul` for a yes/no-style proposition, and `score` if you want a graded severity or quality estimate. The API supports all three question types and returns the selected value, probabilities, `unknown_probability`, and `abstained`.[2] [3]

Python implementation

```
import base64
import json
import requests

API_URL = "http://127.0.0.1:8765/v1/systemone"

def image_data_url(path: str) -> str:
    with open(path, "rb") as f:
        encoded = base64.b64encode(f.read()).decode()
    return f"{encoded}"

def build_request(image_path: str, context: dict) -> dict:
    return {
```

```
        "state": {
            "expense_context": context,
            "document": "<image>"
        },
        "images": [image_data_url(image_path)],
        "questions": {
            "document_type": {
                "type": "choice",
                "instructions": "What type of document is shown?",
                "criteria": {
                    "invoice": None,
                    "receipt": None,
                    "bank_statement": None,
                    "delivery_note": None,
                    "other": None
                }
            },
            "readable": {
                "type": "choice",
                "instructions": "Is the document readable enough for finance process
                "criteria": {
                    "yes": None,
                    "partially": None,
                    "no": None
                }
            },
            "has_total": {
                "type": "noul",
                "instructions": "The document visibly contains a total amount."
            },
            "matches_context": {
                "type": "noul",
                "instructions": "The document is consistent with the submitted vendo
            }
        }
    }

def classify_document(image_path: str, context: dict) -> dict:
    payload = build_request(image_path, context)

    response = requests.post(
        API_URL,
        json=payload,
        timeout=60
    )
    response.raise_for_status()
    return response.json()

context = {
    "employee": "Priya Sharma",
    "vendor": "Office Supplies Ltd",
    "claimed_amount": 184.50,
    "expense_category": "office supplies",
    "submission_id": "EXP-1042"
}
```

```
result = classify_document("receipt.jpg", context)
print(json.dumps(result, indent=2))
```

The exact response field names should be confirmed against your deployed server, but answers are expected to include the chosen value, probability information, `unknown_probability`, and an `abstained` indicator.[2] [3]

Routing rules

```
def route_document(result: dict) -> str:
    answers = result.get("answers", {})

    doc_type = answers.get("document_type", {}).get("value")
    readable = answers.get("readable", {}).get("value")
    has_total = answers.get("has_total", {}).get("value")
    matches = answers.get("matches_context", {}).get("value")

    if readable == "no":
        return "REQUEST_BETTER_IMAGE"

    if doc_type == "invoice":
        return "ACCOUNTS_PAYABLE"

    if doc_type == "receipt":
        if has_total == "yes" and matches == "yes":
            return "EXPENSES_AUTO_QUEUE"
        return "EXPENSES_MANUAL_REVIEW"

    if doc_type == "delivery_note":
        return "PROCUREMENT"

    return "EXCEPTION_QUEUE"
```

Keep the first version conservative: automatically route only clear, low-risk cases; send `unknown`, low-confidence, unreadable, or context-mismatch cases to a human. This matches imajev's intended "act when confident, abstain or defer when not" pattern.[5] [2]

Streamlit interface

```
import streamlit as st

st.title("Invoice & Receipt Intake Bot")

uploaded = st.file_uploader("Upload invoice or receipt", type=["jpg", "jpeg", "png",

vendor = st.text_input("Vendor")
amount = st.number_input("Claimed amount", min_value=0.0)
category = st.selectbox(
    "Expense category",
    ["office supplies", "travel", "software", "meals", "other"]
)
```

```
if uploaded and st.button("Classify and route"):
    with open("uploaded_document.jpg", "wb") as f:
        f.write(uploaded.getvalue())

    context = {
        "vendor": vendor,
        "claimed_amount": amount,
        "expense_category": category
    }

    result = classify_document("uploaded_document.jpg", context)
    st.json(result)
    st.success(f"Routing decision: {route_document(result)}")
```

Database fields to store

Field	Purpose
`submission_id`	Links the document to an expense or AP case
`file_hash`	Helps detect repeated uploads
`document_type`	ImageJev classification
`readable`	Image-quality triage
`has_total`	Basic completeness check
`matches_context`	Consistency with submitted metadata
`confidence`	Model confidence or unknown probability
`route`	Final workflow destination
`reviewer_decision`	Human override for auditability
`created_at`	Reporting and SLA tracking

Corporate enhancements

- **Duplicate detection:** Compare the uploaded file hash and vendor/amount/date against recent submissions before calling the model.
- **Approval thresholds:** Finance can configure, for example, “auto-route only if document type confidence is above 80%.”
- **Reviewer dashboard:** Show unreadable, mismatched, and high-risk documents first.
- **ERP integration:** Push approved records into SAP, Oracle, NetSuite, QuickBooks, or Zoho Books.
- **Audit log:** Preserve the original image, request, response, model version, and reviewer action.
- **Batch intake:** Let finance upload a folder of receipts at month-end and generate a triage report.

Portfolio framing

Present it as: “**An AI document-triage assistant that classifies finance documents, checks submission quality, and routes exceptions to the right team while preserving a human review trail.**” That framing emphasizes business impact—faster intake, fewer miscategorized documents, and better auditability—rather than merely “image classification.” [6] [1]

⁂

1. https://classifier.dev/
2. https://huggingface.co/zenmagnets/Imajev-4B-FP8-SM120
3. https://huggingface.co/mohit67890/imajev-4b
4. https://github.com/fstandhartinger/jevbench/issues/110
5. https://huggingface.co/mohit67890/imajev-9b/raw/f3b3b86e0190328f3b3aa07da11c062cd1e584dd/README.md
6. https://jev101.org/guides/image-decision-model-guide
7. https://github.com/mohit67890/imajev/blob/main/scripts/playground/README.md
8. https://mohit67890.github.io/imajev/report/
9. https://github.com/mohit67890/imajev/blob/main/README.md
10. https://huggingface.co/mohit67890/imajev-9b/blob/f3b3b86e0190328f3b3aa07da11c062cd1e584dd/README.md
11. https://github.com/mohit67890/imajev/blob/main/docs/technical-specification.md
12. https://github.com/mohit67890/imajev/blob/main/docs/imajev-bench-v2-overview.md
13. https://huggingface.co/mohit67890/imajev-9b/blame/f3b3b86e0190328f3b3aa07da11c062cd1e584dd/README.md
14. https://github.com/Shanghua-Gao/RSI-Jev/blob/main/serve/README.md
15. https://github.com/hotchpotch/S1MB/blob/main/docs/vision-evaluation.md
16. https://github.com/mohit67890/imajev